

Compiler Construction

Übung 01 - Grammatiken

Aufgabe 01 - Grundbegriffe

a.) Wie lang (in Terminalsymbolen) ist der kürzeste Satz in MicroJava, der mindestens ein *Tilde* ("~") enthält? Geben Sie ein Beispiel für einen Satz dieser Länge an. Der Satz muss nur syntaktisch korrekt (entspricht der Grammatik) sein, nicht semantisch (hält alle Kontextbedingungen der Sprache ein). Beispielsweise ist $x = x + 1$, ohne x zuvor zu definieren, eine syntaktisch korrekte Zuweisung, semantisch korrekt ist sie aber nicht.

Die benötigten "Komponenten", um so einen Satz zusammenzubauen finden sich in:

- Program
- MethodDecl
- Block
- Statement
- Designator

Der "Pfad" sieht so aus:

```
"program" ident "{" {MethodDecl} "}" // Program
"void" ident "(" ")" Block // MethodDecl
"{" {Statement} "}" // Block
Designator "++" ";" // Statement
ident {"[" "~" number "]"}
```

Das ganze "ausprogrammiert" würde dann so aussehen:

```
program P {
    void method() {
        x[~1]++;
    }
}
```

Es werden also insgesamt 17 Terminalsymbole benötigt.

b.) Sind die Nonterminalsymbole *Statement* und *ActPars* rekursiv? Wenn ja, geben Sie alle Rekursionsarten an (Transitivität: direkt, indirekt; Orientierung: links, rechts, zentral). Beispiele für Rekursionsarten sind direkt zentralrekursiv, indirekt linksrekursiv, . . . Pro NTS kann es mehrere Rekursionsarten geben. Geben Sie bei indirekten Rekursionen einen möglichen Pfad an (Beispiel: Factor ist indirekt zentralrekursiv über Factor -> Expr -> Term -> Factor).

Statement:

direkt rechtsrekursiv:

- `Statement -> "if" "(" Condition ")" Statement`
- `Statement -> "while" "(" Condition ")" Statement`

direkt zentralrekursiv:

- `Statement -> "if" "(" Condition ")" Statement "else" Statement` (aufgrund des ersten "Statement")

indirekt zentralrekursiv:

- `Statement -> Block -> "{" {Statement} "}"`
 - Pfad: `Statement -> Block -> Statement`

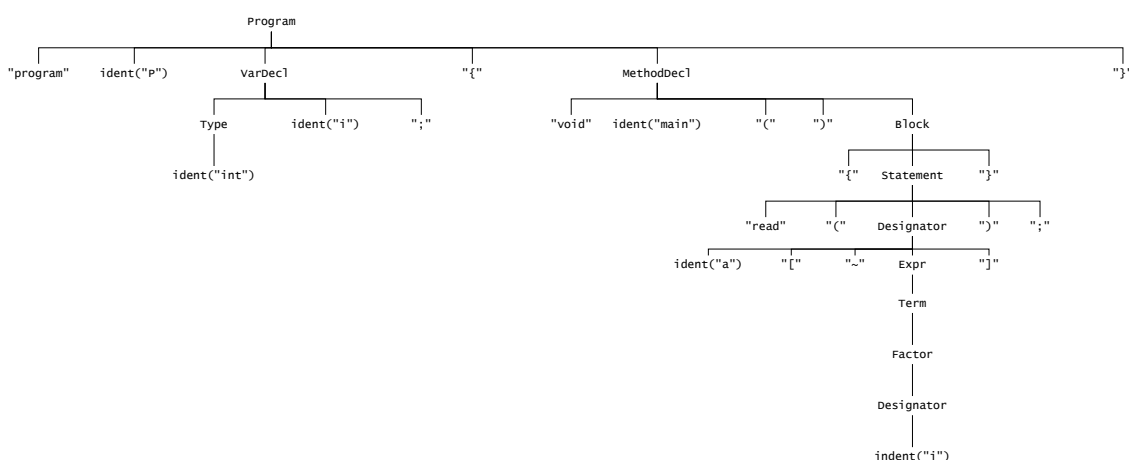
ActPars:

indirekt rechtsrekursiv:

- `ActPars -> Expr -> Term -> Factor -> Designator [ActPars]`

c.) Wie sieht der konkrete Syntaxbaum für folgenden Satz aus? Gibt es mehrere konkrete Syntaxbäume für diesen Satz? Geben Sie für Symbole der Terminalklassen *ident* und *charConst* im Baum auch die zugehörigen Werte an. Beispiel: *ident* ("main")

```
program P int i; { void main(){ read(a[~i]); }}
```



d.) Welche terminalen Anfänge und Nachfolger haben die Regeln *MethodDecl*, *AssignOp* und *CondTerm*? Bitte die finalen Antworten komplett ausschreiben (i.e., jeweils ein Set an TS)

MethodDecl:

- `First(MethodDecl) = First(Type), "void", "(" = ident, "void", "("`
- `Follow(MethodDecl) = "}"`

AssignOp

- `First(AssignOp) = "=", "+=", "-=", "*=", "/=", "%="`
- `Follow(AssignOp) = First(Expr)
= First(Term)
= First(Factor)
= First(Designator), number, charConst, "new" ident, "("
= ident, number, charConst, "new" ident, "("`

CondTerm

- `First(CondTerm) = First(CondFact)
= First(Expr)
= ident, number, charConst, "new" ident, "("`
- `Follow(CondTerm) = "||", Follow(Condition)
= "||", ")"`

Aufgabe 02 - Konstruktion einer Grammatik

Geben Sie eine EBNF-Grammatik an, die folgende Beschreibung abdeckt. Es handelt sich dabei um eine fiktive Schachnotation. Bilden Sie dabei beliebig viele Non-Terminalklassen, mindestens jedoch eine für jedes **fett** geschriebene Wort.

Ein **Spiel** setzt sich aus mindestens einem **Zug** zusammen, wobei Züge durch Beistriche (",") getrennt sind. Nach dem letzten **Zug** steht ein Strichpunkt (";").

Ein **Zug** beginnt mit einer **Figur**, gefolgt von einer **Position** (Startposition). Darauf folgt ein Bindestrich ("-"), gefolgt von einer weiteren **Position** (Zielposition). Sollte durch den **Zug** eine **Figur** geschlagen worden sein, folgt ein "x". Unabhängig davon, ob eine **Figur** geschlagen wurde, folgt gegebenenfalls die Information, ob der Gegner Schach (+) oder Schach-Matt (#) steht. Ein Spieler kann nicht gleichzeitig Schach und Schach-Matt stehen.

Beispielzüge: Ka1-a2, Qf5-h7x#, Pb2-c3x, Rh1-h6+, Bb5-d7x+

Eine **Figur** ist entweder ein König ("K"), eine Königin ("Q"), ein Turm ("R"), ein Läufer ("B"), ein Springer ("N"), oder ein Bauer ("P").

Eine **Position** ist eine **Spalte**, gefolgt von einer **Zeile**.

Eine **Spalte** ist ein Buchstabe zwischen a und h (a, b, c, d, e, f, g, h).

Eine **Zeile** ist eine Zahl zwischen 1 und 8 (1, 2, 3, 4, 5, 6, 7, 8).

```
Game = Move {",", Move} ";"  
Move = ChessPiece Position "-" Position ["x"] ["+" | "#"]  
ChessPiece = "K" | "Q" | "R" | "B" | "N" | "P"  
Position = Column Row  
Column = "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"  
Row = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8"
```

Aufgabe 03 - Beseitigung von Linksrekursionen

```
Object = ident ":" "{" Props "}" ";".  
Props = ident ":" Value | Props "," ident ":" Value .  
Value = "\"" text "\"" | Value "+" "\"" text "\"" | ε.
```

Beispielsätze:

```
alice : { name: 'Alice', greeting: 'welcome' + 'alice' };  
bob: { name: 'Bob', greeting: 'hello' + 'to' + 'bob' };  
test: { noValue: , emptyValue: 'empty', plusValue: + 'plus' };
```

a.) Wo befinden sich in der folgenden Grammatik Linksrekursionen? Geben Sie die Produktionen an.

- `Props = Props "," ident ":" Value.`
- ``Value = Value "+" "" text ""`.`

b.) Wie könnte man die Linksrekursionen entfernen? Geben Sie eine umgeformte EBNF Grammatik ohne Linksrekursionen an. Ersetzen Sie die Linksrekursionen dabei durch Iterationen.

- `Props = ident ":" Value {"," ident ":" Value}.`
 - `Value = ["" text "" ] { "+" "" text "" }.`
-